

Institut National des Postes et Télécommunications

Adversarial Machine Learning

Appliqué aux Systèmes de Détection d’Intrusion

Red Teaming & Blue Teaming d’un IDS basé sur un réseau
de neurones *MLP* sous PyTorch

Red Teaming



Blue Teaming

Sécurité des modèles de Deep Learning pour les IDS

Auteur(s) :	Ayman MIDAN Mohamed EL HARCHALI
Encadrant :	Pr. Tarik FISSAA
Module :	Deep Learning
Dataset :	UNSW-NB15
Année universitaire :	2025–2026

Table des matières

1	Introduction	2
2	Méthodologie : Données, Modèle et Évaluation	2
2.1	Le Dataset UNSW-NB15	2
2.1.1	Présentation Générale	2
2.1.2	Pré-traitement	2
2.1.3	Mapping des Contraintes Physiques	3
2.2	Architecture du Modèle Baseline	3
2.2.1	Définition Formelle	3
2.2.2	Configuration d’Entraînement	3
2.3	Métriques d’Évaluation	3
2.3.1	Métriques de Classification	3
2.3.2	Attack Success Rate (ASR)	4
2.4	Performances du Modèle Baseline	4
3	Red Teaming : Audit Adversarial de l’IDS	4
3.1	Philosophie du Red Teaming en Sécurité ML	4
3.2	Attaque 1 – Boîte Blanche : CPGD	4
3.2.1	Formalisation Mathématique	4
3.2.2	L’Opération de Projection : Cœur de l’Approche	5
3.2.3	Pseudo-code	5
3.2.4	Résultats et Analyse	5
3.3	Attaque 2 – Boîte Noire : PSO	6
3.3.1	Motivation : Au-delà du Gradient	6
3.3.2	Formalisation Mathématique	6
3.3.3	Pseudo-code	7
3.3.4	Résultats et Analyse	7
3.4	Comparaison des Deux Vecteurs d’Attaque	7
4	Blue Teaming : Entraînement Adversarial Robuste	8
4.1	Stratégie Défensive	8
4.1.1	Principe de l’Entraînement Adversarial	8
4.1.2	Implémentation : CPGD Rapide Intra-Boucle	8
4.1.3	Pseudo-code de la Boucle d’Entraînement Robuste	8
4.2	Évaluation Post-Défense	8
4.2.1	Résistance au CPGD (Attaque d’Entraînement)	9
4.2.2	Résistance Croisée au PSO (Robustesse de Transfert)	9
4.3	Tableau de Bord Récapitulatif	9
5	Discussion Critique	9
5.1	Analyse du Paradoxe Boîte Blanche / Boîte Noire	9
5.2	Limites et Biais Potentiels	10
6	Conclusion et Perspectives	10
6.1	Synthèse des Contributions	10
6.2	Perspectives de Recherche	10

1 Introduction

L’évolution rapide des infrastructures numériques (*cloud*, IoT, réseaux distribués) a considérablement élargi la surface d’attaque des systèmes informatiques. Dans ce contexte, les **Systèmes de Détection d’Intrusion** (IDS) jouent un rôle essentiel en analysant le trafic réseau afin d’identifier les comportements malveillants en temps réel.

Les approches modernes basées sur le *Deep Learning* ont permis d’améliorer significativement les performances des IDS grâce à leur capacité à apprendre automatiquement des représentations complexes du trafic réseau. Toutefois, ces modèles restent vulnérables aux **attaques adversariales**, où de faibles perturbations des données d’entrée suffisent à provoquer une mauvaise classification.

Dans le domaine réseau, la génération d’exemples adversariaux présente une contrainte supplémentaire : les modifications doivent rester **physiquement réalisables**. Contrairement aux images, certaines caractéristiques réseau ne peuvent pas être modifiées arbitrairement après transmission. Une attaque crédible doit donc respecter les contraintes causales du trafic réel.

Ce projet étudie la robustesse d’un IDS basé sur un réseau de neurones MLP entraîné sur le dataset **UNSW-NB15**. L’approche adoptée suit un cycle complet :

1. conception d’un modèle Baseline performant ;
2. évaluation offensive (*Red Teaming*) via des attaques adversariales contraintes (CPGD et PSO) ;
3. renforcement défensif (*Blue Teaming*) par entraînement adversarial.

L’objectif principal est d’évaluer dans quelle mesure un IDS basé sur le Deep Learning reste fiable face à des attaques réalistes respectant les contraintes physiques du réseau.

2 Méthodologie : Données, Modèle et Évaluation

2.1 Le Dataset UNSW-NB15

2.1.1 Présentation Générale

Le jeu de données **UNSW-NB15** a été généré par le Cyber Range Lab de l’Université de Nouvelle-Galles du Sud à Sydney en 2015. Il constitue aujourd’hui une référence académique pour l’évaluation des IDS *machine learning*. Ses caractéristiques principales sont résumées dans le tableau 1.

TABLE 1 – Caractéristiques du dataset d’entraînement UNSW-NB15

Attribut	Valeur
Nombre d’instances	≈ 82,5 milles
Nombre de features brutes	49
Features sélectionnées	42 (après suppression des champs non-numériques et identifiants)
Classes	Binaire : Bénin (0) / Attaque (1)
Familles d’attaques	DoS, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode, Worms, Backdoor, Analysis
Equilibre de classes	≈ 51 % bénin / 49 % attaque

2.1.2 Pré-traitement

La pipeline de pré-traitement applique les étapes suivantes :

1. **Sélection de features** : suppression des colonnes non-numériques (**proto**, **service**, **state**), des identifiants et horodatages. On obtient $d = 42$ features continues.
2. **Normalisation** : standardisation z -score feature par feature, $\tilde{x}_j = (x_j - \mu_j)/\sigma_j$, les statistiques étant calculées *uniquement* sur le Training Set pour éviter toute fuite de données.
3. **Partitionnement** : split stratifié 80 % / 20 % (train/test), préservant le ratio de classes.

2.1.3 Mapping des Contraintes Physiques

Pour les attaques adversariales, les 42 features sont partitionnées en deux groupes :

- **Features unilatérales** (augmentables uniquement) : **dur** (durée), **spkts**, **dpkts** (nombres de paquets), **sbytes**, **dbytes** (octets échangés), **sload**, **dload** (bande passante), etc. – *tout ce qui implique du temps écoulé ou des données émises*.
- **Features libres** : ratios, statistiques dérivées modifiables dans les deux sens sans violer la causalité physique.

2.2 Architecture du Modèle Baseline

2.2.1 Définition Formelle

Le modèle est un **Perceptron Multi-Couches** (MLP) PyTorch de faible profondeur, choisi pour sa combinaison de performances élevées et d’interprétabilité relative sur données tabulaires.

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{64 \times 42} \quad (1)$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2), \quad \mathbf{W}_2 \in \mathbb{R}^{32 \times 64} \quad (2)$$

$$\hat{y} = \mathbf{w}_3^\top \mathbf{h}_2 + b_3, \quad \mathbf{w}_3 \in \mathbb{R}^{32} \quad (3)$$

La sortie $\hat{y}$ est un logit non borné. La fonction de perte utilisée est la **Binary Cross-Entropy with Logits Loss** :

$$\mathcal{L}_{\text{BCE}}(\hat{y}, y) = -[y \cdot \log \sigma(\hat{y}) + (1 - y) \cdot \log(1 - \sigma(\hat{y}))] \quad (4)$$

où $\sigma(\cdot)$ est la fonction sigmoïde. La classification s’effectue par seuillage à $\sigma(\hat{y}) \geq 0.5$.

2.2.2 Configuration d’Entraînement

TABLE 2 – Hyperparamètres du modèle Baseline

Hyperparamètre	Valeur
Optimiseur	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Learning rate	10^{-3}
Taille de batch	256
Nombre d’époques	20
Dropout	Aucun
Weight decay	10^{-4}

2.3 Métriques d’Évaluation

2.3.1 Métriques de Classification

Soit TP, TN, FP, FN les effectifs de la matrice de confusion :

$$\text{Précision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (5)$$

$$\text{Rappel} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (6)$$

$$\text{F1} = \frac{2 \cdot \text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (7)$$

2.3.2 Attack Success Rate (ASR)

Pour les expériences adversariales, l’**Attack Success Rate** mesure le taux de faux négatifs induits par l’attaque sur un sous-ensemble d’exemples malveillants correctement classifiés à l’origine :

$$\text{ASR} = \frac{|\{\mathbf{x} \in \mathcal{D}_{\text{atk}} : f(\mathbf{x}) = 1 \wedge f(\mathbf{x}^{\text{adv}}) = 0\}|}{|\{\mathbf{x} \in \mathcal{D}_{\text{atk}} : f(\mathbf{x}) = 1\}|} \quad (8)$$

Un ASR de 100 % signifie que l’IDS ne détecte plus aucune attaque.

2.4 Performances du Modèle Baseline

Résultats – Modèle Naïf (Baseline)

Métrique	Valeur	Interprétation
Précision	98.50 %	Très faible taux de fausses alarmes
Rappel	85.46 %	Détecte 85 attaques sur 100
F1-Score	91.52 %	Excellent équilibre général

Le Rappel de 85.46 % révèle un **angle mort structurel** : 14.54 % des attaques échappent déjà au modèle, sans intervention adversariale. Ce défaut initial sera amplifié par les attaques décrites à la section suivante.

3 Red Teaming : Audit Adversarial de l’IDS

3.1 Philosophie du Red Teaming en Sécurité ML

Le *Red Teaming* consiste à adopter le point de vue de l’adversaire pour identifier proactivement les failles d’un système avant qu’elles ne soient exploitées en production. Dans le contexte de l’Adversarial ML, l’objectif est de générer des **exemples adversariaux** $\mathbf{x}^{\text{adv}}$ satisfaisant simultanément :

$$\mathbf{x}^{\text{adv}} = \arg \max_{\mathbf{x}'} \mathcal{L}(f(\mathbf{x}'), y_{\text{atk}}) \quad \text{s.t.} \quad \mathbf{x}^{\text{adv}} \in \mathcal{C}(\mathbf{x}) \quad (9)$$

où $\mathcal{C}(\mathbf{x})$ est l’ensemble des perturbations physiquement valides à partir de $\mathbf{x}$, et $y_{\text{atk}} = 1$ (le vrai label d’une attaque que l’on cherche à faire passer pour bénigne). Nous employons deux paradigmes d’attaque complémentaires : la **boîte blanche** (accès complet au modèle, ses poids et gradients) et la **boîte noire** (accès limité aux prédictions en sortie).

3.2 Attaque 1 – Boîte Blanche : CPGD

3.2.1 Formalisation Mathématique

Le **Constrained Projected Gradient Descent** (CPGD) est une adaptation de l’attaque PGD de Madry *et al.* [2] intégrant une **opération de projection** $\mathcal{P}_{\mathcal{C}}$ qui garantit la validité physique à chaque itération. L’objectif est de maximiser la perte :

$$\mathbf{x}^{\text{adv}} = \arg \max_{\mathbf{x}' \in \mathcal{C}(\mathbf{x})} \mathcal{L}_{\text{BCE}}(f(\mathbf{x}'), y = 1) \quad (10)$$

L’algorithme itératif procède ainsi pour chaque itération t :

$$\begin{aligned} \mathbf{g}^{(t)} &= \nabla_{\mathbf{x}} \mathcal{L}_{\text{BCE}}(f(\mathbf{x}^{(t)}), 1) && \text{(Gradient ascendant)} \\ \tilde{\mathbf{x}}^{(t+1)} &= \mathbf{x}^{(t)} + \epsilon \cdot \text{sign}(\mathbf{g}^{(t)}) && \text{(Pas FGSM)} \\ \mathbf{x}^{(t+1)} &= \mathcal{P}_{\mathcal{C}}(\tilde{\mathbf{x}}^{(t+1)}) && \text{(Projection physique)} \end{aligned}$$

3.2.2 L’Opération de Projection : Cœur de l’Approche

La projection $\mathcal{P}_C$ est l’élément différenciateur central de ce travail. Pour chaque feature j , elle est définie par :

$$[\mathcal{P}_C(\tilde{\mathbf{x}})]_j = \begin{cases} \max(\tilde{x}_j, x_j) & \text{si la feature } j \text{ est unilatérale (temps, taille)} \\ \text{clip}(\tilde{x}_j, x_j - \delta_j, x_j + \delta_j) & \text{sinon} \end{cases} \quad (11)$$

Pourquoi cette Contrainte est Cruciale

En vision par ordinateur, un pixel peut être assombri ou éclairci librement ; la contrainte est simplement une boule ℓ_∞ . L’exemple adversarial reste une image visuellement cohérente.

En cybersécurité réseau, réduire la durée d’une session TCP déjà terminée, ou diminuer le nombre d’octets envoyés, est *causalement impossible* : ces événements sont dans le passé. Un générateur d’attaques qui produirait **dur** = -5s génère un exemple non-réalisable, sans valeur pratique.

La contrainte $\max(\tilde{x}_j, x_j)$ modélise le fait que l’attaquant peut **ajouter du délai** (injecter des paquets de *keep-alive*) ou **ajouter du padding** à sa charge utile, mais ne peut pas réécrire l’histoire réseau. Ceci rend nos exemples adversariaux **directement exploitables** par un vrai attaquant.

3.2.3 Pseudo-code

Algorithm 1 CPGD – Constrained Projected Gradient Descent

Require: Échantillon malveillant $\mathbf{x}$, modèle f , pas ϵ , itérations T , masque de contraintes $\mathbf{m}$ (1 = unilatéral)

Ensure: Exemple adversarial $\mathbf{x}^{\text{adv}}$ physiquement valide

```

1:  $\mathbf{x}^{(0)} \leftarrow \mathbf{x}$ 
2: for  $t = 0, 1, \dots, T - 1$  do
3:    $\mathbf{g}^{(t)} \leftarrow \nabla_{\mathbf{x}} \mathcal{L}_{\text{BCE}}(f(\mathbf{x}^{(t)}), 1)$ 
4:    $\tilde{\mathbf{x}}^{(t+1)} \leftarrow \mathbf{x}^{(t)} + \epsilon \cdot \text{sign}(\mathbf{g}^{(t)})$ 
5:   for  $j = 1, \dots, d$  do
6:     if  $m_j = 1$  then ▷ Feature unilatérale
7:        $x_j^{(t+1)} \leftarrow \max(\tilde{x}_j^{(t+1)}, x_j^{(0)})$ 
8:     else
9:        $x_j^{(t+1)} \leftarrow \text{clip}(\tilde{x}_j^{(t+1)}, x_j^{(0)} - \delta_j, x_j^{(0)} + \delta_j)$ 
10:    end if
11:  end for
12: end for return  $\mathbf{x}^{(T)}$ 
```

3.2.4 Résultats et Analyse

Résultat – Attaque CPGD (Boîte Blanche)

Attack Success Rate : 53.23 %

L’IDS est trompé sur plus d’une attaque sur deux.

Un ASR de 53.23 % sur un modèle réputé robuste est un résultat significatif. Il démontre que le **paysage de perte du MLP n’est pas convexe** dans les directions de gradient contraint. La projection réduit l’espace explorable, ce qui explique que l’ASR reste inférieur à 100 % : certaines attaques n’ont pas de voisinage physiquement valide qui franchisse la frontière de décision. Néanmoins, l’existence même de cet ASR révèle des **zones de vulnérabilité structurelle** dans l’espace des features réseau.

3.3 Attaque 2 – Boîte Noire : PSO

3.3.1 Motivation : Au-delà du Gradient

La CPGD exploite une information privilégiée : le gradient $\nabla_{\mathbf{x}}\mathcal{L}$. Un attaquant réel n’a généralement accès qu’à l’**entrée** et à la **sortie** de l’IDS (un score ou une décision binaire), sans connaître l’architecture ni les poids. Il convient donc de tester la robustesse du modèle face à une attaque purement heuristique, *gradient-free*.

La **Particle Swarm Optimization** (PSO) [?] est un algorithme évolutionnaire inspiré du comportement collectif des nuées d’oiseaux. Dans notre contexte, chaque “particule” représente un exemple adversarial candidat, et l’essaim converge collectivement vers les angles morts de l’IDS.

3.3.2 Formalisation Mathématique

L’essaim est constitué de $N = 30$ particules $\{\mathbf{x}_i\}_{i=1}^N$. Chaque particule i possède une position $\mathbf{x}_i^{(t)} \in \mathcal{C}(\mathbf{x})$ et une vitesse $\mathbf{v}_i^{(t)}$. La mise à jour est régie par :

$$\mathbf{v}_i^{(t+1)} = \omega \mathbf{v}_i^{(t)} + c_1 r_1 \underbrace{(\mathbf{p}_i^{\text{best}} - \mathbf{x}_i^{(t)})}_{\text{mémoire individuelle (pBest)}} + c_2 r_2 \underbrace{(\mathbf{g}^{\text{best}} - \mathbf{x}_i^{(t)})}_{\text{mémoire collective (gBest)}} \quad (12)$$

$$\mathbf{x}_i^{(t+1)} = \mathcal{P}_{\mathcal{C}}\left(\mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)}\right) \quad (13)$$

où ω est le facteur d’inertie, c_1, c_2 sont les coefficients d’accélération cognitif et social, et $r_1, r_2 \sim \mathcal{U}(0, 1)$ sont des tirages aléatoires. La **fonction d’aptitude** (*fitness*) minimisée par chaque particule est :

$$\text{fitness}(\mathbf{x}_i) = \sigma(f(\mathbf{x}_i)) \in [0, 1] \quad (14)$$

Une particule “réussit” si $\text{fitness}(\mathbf{x}_i) < 0.5$ (l’IDS classe l’attaque comme bénigne).

TABLE 3 – Hyperparamètres du PSO

Paramètre	Symbole	Valeur
Taille de l’essaim	N	30
Nombre d’itérations	T	40
Facteur d’inertie	ω	0.7
Coefficient cognitif	c_1	1.5
Coefficient social	c_2	1.5

3.3.3 Pseudo-code

Algorithm 2 PSO – Particle Swarm Optimization Contrainte

Require: Échantillon $\mathbf{x}$, modèle f (boîte noire), N , T , masque $\mathbf{m}$

```

1: Initialisation :  $\mathbf{x}_i^{(0)} \sim \mathcal{U}(\mathbf{x}, \mathbf{x} + \delta)$ ,  $\mathcal{P}_C$  appliquée,  $\mathbf{v}_i^{(0)} = \mathbf{0}$ 
2:  $\mathbf{p}_i^{\text{best}} \leftarrow \mathbf{x}_i^{(0)}$  pour tout  $i$ 
3:  $\mathbf{g}^{\text{best}} \leftarrow \arg \min_i \sigma(f(\mathbf{x}_i^{(0)}))$ 
4: for  $t = 0, 1, \dots, T - 1$  do
5:   for  $i = 1, \dots, N$  do
6:     Mise à jour de  $\mathbf{v}_i^{(t+1)}$  selon (12)
7:      $\tilde{\mathbf{x}}_i^{(t+1)} \leftarrow \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)}$ 
8:      $\mathbf{x}_i^{(t+1)} \leftarrow \mathcal{P}_C(\tilde{\mathbf{x}}_i^{(t+1)})$ 
9:     if  $\sigma(f(\mathbf{x}_i^{(t+1)})) < \sigma(f(\mathbf{p}_i^{\text{best}}))$  then
10:       $\mathbf{p}_i^{\text{best}} \leftarrow \mathbf{x}_i^{(t+1)}$ 
11:    end if
12:  end for
13:   $\mathbf{g}^{\text{best}} \leftarrow \arg \min_i \sigma(f(\mathbf{p}_i^{\text{best}}))$ 
14:  if  $\sigma(f(\mathbf{g}^{\text{best}})) < 0.5$  then break
15:  end if
16: end for return  $\mathbf{g}^{\text{best}}$ 

```

3.3.4 Résultats et Analyse

Résultat – Attaque PSO (Boîte Noire)

Attack Success Rate : 97.95 %

L’essai contourne l’IDS sur près de 98 attaques sur 100.

Ce résultat est **remarquable et contre-intuitif** : une attaque en boîte noire, sans accès aux gradients, est *presque deux fois plus efficace* que l’attaque en boîte blanche. Cette observation révèle des informations structurelles importantes sur le modèle :

1. **Non-convexité profonde du paysage de perte** : la CPGD suit le gradient localement et peut se retrouver piégée dans des maxima locaux de perte non-suffisants pour franchir la frontière de décision. Le PSO, en explorant l’espace de manière stochastique et décentralisée, découvre des **bassins adversariaux** inaccessibles au gradient contraint.
2. **Présence de “angles morts” non-linéaires** : le MLP apprend des frontières de décision complexes avec des régions de confiance élevée entourées de “gouffres” adversariaux. Ces structures non-convexes sont précisément celles que la recherche par essaim sait exploiter.
3. **Transfert de menace réaliste** : un attaquant sans connaissance de l’architecture (scénario commercial, API tierce partie) peut néanmoins compromettre l’IDS avec un taux de succès quasi-total.

3.4 Comparaison des Deux Vecteurs d’Attaque

TABLE 4 – Comparaison Red Teaming – CPGD vs PSO

Attaque	Paradigme	Connaissance	ASR	Complexité
CPGD	Gradient	Boîte blanche	53.23 %	$\mathcal{O}(T \cdot d)$
PSO	Essaim	Boîte noire	97.95 %	$\mathcal{O}(T \cdot N \cdot d)$

La divergence entre ces deux **ASR** constitue un signal d’alarme pour le déploiement en production : l’attaque la plus accessible (boîte noire, sans ingénierie inverse du modèle) est aussi la plus dévastatrice.

4 Blue Teaming : Entraînement Adversarial Robuste

4.1 Stratégie Défensive

4.1.1 Principe de l’Entraînement Adversarial

L’entraînement adversarial (*Adversarial Training*) [2] est la méthode de défense empirique la plus établie en Adversarial ML. L’intuition fondamentale est d’**augmenter le jeu de données d’entraînement** avec des exemples adversariaux, forçant le modèle à apprendre des frontières de décision robustes aux perturbations.

Formellement, l’objectif d’entraînement standard :

$$\min_{\theta} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\mathcal{L}(f_{\theta}(\mathbf{x}), y)] \quad (15)$$

est remplacé par l’objectif min-max de Madry :

$$\min_{\theta} \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[\max_{\mathbf{x}' \in \mathcal{C}(\mathbf{x})} \mathcal{L}(f_{\theta}(\mathbf{x}'), y) \right] \quad (16)$$

Le problème interne (max) génère les exemples les plus difficiles ; le problème externe (min) entraîne le modèle à y résister.

4.1.2 Implémentation : CPGD Rapide Intra-Boucle

Pour des raisons d’efficacité computationnelle, le problème interne est approximé par un CPGD de **5 itérations** (au lieu des 20+ de l’audit), suffisant pour générer des exemples diversifiés et représentatifs. Cette approche, parfois appelée **Free Adversarial Training** [?] dans sa variante la plus agressive, maintient un surcoût d’entraînement modéré.

Garantie d’Intégrité : Absence de Fuite de Données

La génération d’exemples adversariaux s’effectue **exclusivement sur le Training Set**. Le Test Set n’est jamais exposé pendant l’entraînement, garantissant la validité des métriques d’évaluation post-défense. Cette discipline est fondamentale : un adversarial training qui “contaminerait” le test set produirait des métriques de robustesse artificiellement optimistes.

4.1.3 Pseudo-code de la Boucle d’Entraînement Robuste

Algorithm 3 Boucle d’Entraînement Adversarial

Require: Training Set $\mathcal{D}_{\text{train}}$, taille de batch B , époque E , CPGD rapide $\text{CPGD}_{K=5}$, optimiseur Adam

- 1: **for** époque $e = 1, \dots, E$ **do**
- 2: **for** chaque mini-batch $(\mathbf{X}_b, \mathbf{y}_b)$ de taille B **do**
- 3: $\mathbf{X}_b^{\text{adv}} \leftarrow \text{CPGD}_{K=5}(\mathbf{X}_b, f_{\theta})$ ▷ Génération adversariale
- 4: $\tilde{\mathbf{X}} \leftarrow \text{concat}(\mathbf{X}_b, \mathbf{X}_b^{\text{adv}})$
- 5: $\tilde{\mathbf{y}} \leftarrow \text{concat}(\mathbf{y}_b, \mathbf{y}_b)$ ▷ Les labels restent inchangés
- 6: $\mathcal{L} \leftarrow \frac{1}{2B} \sum_i \mathcal{L}_{\text{BCE}}(f_{\theta}(\tilde{x}_i), \tilde{y}_i)$
- 7: $\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}$ ▷ Descente de gradient
- 8: **end for**
- 9: **end for** **return** f_{θ} (modèle vacciné)

4.2 Évaluation Post-Défense

4.2.1 Résistance au CPGD (Attaque d’Entraînement)

Résultat – Robustesse CPGD après Vaccination

ASR_{CPGD} : 53.23 % $\rightarrow$ 2.41 %

Réduction de **-95.5 %**. Le modèle vacciné est quasi-imperméable à l’attaque dont il a appris les patterns.

La chute de l’ASR à 2.41 % confirme que l’entraînement adversarial est **extrêmement efficace** contre le vecteur d’attaque qu’il a intégré. Le modèle a appris à reconnaître les patterns de perturbation du CPGD et a développé des frontières de décision orthogonales aux directions de gradient adversarial.

4.2.2 Résistance Croisée au PSO (Robustesse de Transfert)

Résultat – Robustesse Croisée PSO après Vaccination

ASR_{PSO} : 97.95 % $\rightarrow$ 49.61 %

Réduction de **-49.3 %**. L’essai voit son efficacité divisée par deux, sans avoir été utilisé pendant la défense.

Cette **robustesse croisée** est le résultat le plus intéressant du projet. Sans avoir jamais exposé le modèle à des exemples PSO pendant l’entraînement, la vaccination contre le CPGD confère une résistance significative au PSO. Ce phénomène s’explique par le mécanisme suivant : l’entraînement adversarial CPGD oblige le modèle à apprendre des **représentations plus généralisables** dans les régions adversariales. La frontière de décision, rendue plus robuste aux perturbations de gradient, est également plus robuste aux perturbations évolutives, qui cherchent les mêmes angles morts mais par un chemin différent.

Néanmoins, un ASR résiduel de 49.61 % pour le PSO rappelle que la robustesse obtenue est **incomplète**, ainsi, la défense asymptotique requiert des techniques complémentaires décrites en section Perspectives.

4.3 Tableau de Bord Récapitulatif

TABLE 5 – Synthèse complète des résultats – Baseline, Red Teaming, Blue Teaming

Phase	Métrique	Modèle Naïf	Modèle Vacciné
Classification	Précision	98.50 %	$\approx$ 97.8 %
	Rappel	85.46 %	$\approx$ 88.2 %
	F1-Score	91.52 %	$\approx$ 92.8 %
Red Teaming	ASR – CPGD (BB)	53.23 %	2.41 %
	ASR – PSO (BN)	97.95 %	49.61 %

Observation : Robustesse sans Dégradation Significative

Un effet classique et indésirable de l’entraînement adversarial est la **dégradation des performances sur données propres** (*accuracy drop*). Dans notre expérience, ce compromis est très favorable : la vaccination améliore légèrement le Rappel (détection d’attaques classiques améliorée) tout en maintenant une Précision élevée. Cette observation suggère que les exemples adversariaux contraignent le jeu d’entraînement de manière constructive.

5 Discussion Critique

5.1 Analyse du Paradoxe Boîte Blanche / Boîte Noire

Le résultat le plus surprenant de cette étude – la supériorité de l’attaque en boîte noire (PSO, 97.95 %) sur l’attaque en boîte blanche (CPGD, 53.23 %) – mérite une analyse approfondie.

Ce paradoxe apparent s’explique par la **géométrie du paysage de perte contraint**. La CPGD suit le gradient local, qui dans l’espace contraint peut pointer vers des directions non-productives (le gradient “naturel” est partiellement annulé par la projection $\mathcal{P}_C$). En revanche, le PSO explore de manière stochastique et peut franchir des barrières de perte que le gradient ne peut pas localiser.

Ce phénomène est connu dans la littérature comme le **problème du gradient masqué** (*gradient masking*) : certaines défenses – ici involontairement – créent des gradients peu informatifs, donnant une fausse impression de robustesse aux attaques de type gradient. Un audit en boîte noire est donc **indispensable** pour toute évaluation sérieuse.

5.2 Limites et Biais Potentiels

1. **UNSW-NB15 et dérive temporelle** : le dataset date de 2015. Les patterns d’attaques modernes (attaques par canal caché, exfiltration via DNS/HTTPS) peuvent ne pas être représentés. Un IDS entraîné sur ces données peut être aveugle à des menaces contemporaines.
2. **Feature engineering partiel** : les 42 features sélectionnées excluent les champs catégoriels (protocole, service). Une représentation plus riche incluant des embeddings de ces champs pourrait modifier la topologie adversariale.
3. **PSO non intégré à la défense** : la robustesse croisée obtenue (49.61 %) laisse un risque résiduel substantiel. Un entraînement adversarial mixte (CPGD + PSO) serait l’étape logique suivante.
4. **Hypothèse d’un attaquant unique** : notre modèle suppose un attaquant optimisant chaque paquet individuellement. Des attaques coordonnées (botnets) ou des attaques adaptatives (qui modifient leur stratégie en fonction de la réponse de l’IDS) ne sont pas modélisées.

6 Conclusion et Perspectives

6.1 Synthèse des Contributions

Ce projet a démontré le cycle complet **Build – Break – Fix** d’un système d’intelligence artificielle critique :

1. **Construction d’un IDS performant** ($F1 = 91.52\%$) sur UNSW-NB15, avec une pipeline de pré-traitement rigoureuse et une validation sans fuite de données.
2. **Développement d’attaques physiquement contraintes**, distinguant ce travail des approches naïves en vision : la projection $\mathcal{P}_C$ garantit la réalisabilité opérationnelle des exemples adversariaux.
3. **Mise en évidence de la supériorité paradoxale de la boîte noire** (PSO à 97.95 % vs CPGD à 53.23 %), révélant les limites du gradient contraint face aux paysages de perte non-convexes.
4. **Vaccination efficace** par entraînement adversarial, réduisant l’ASR du CPGD à 2.41 % et offrant une robustesse croisée significative au PSO (49.61 %).

6.2 Perspectives de Recherche

Court terme.

- **Entraînement adversarial mixte** : intégrer des exemples PSO dans la boucle de défense pour combler le risque résiduel de 49.61 %.
- **Randomized Smoothing** : ajouter une couche de certification probabiliste permettant de borner formellement l’ASR.

Moyen terme.

- **Transfert vers des IDS multi-étiquettes** : étendre le cadre aux 9 catégories d’attaques d’UNSW-NB15 pour évaluer la robustesse adversariale par famille.
- **Attaques adaptatives** : implémenter un attaquant qui connaît la stratégie de défense et la contourne, constituant le test de robustesse le plus sévère.

Long terme.

- **Déploiement sur données de trafic réel** (PCAP) avec extraction automatique de features, pour valider la chaîne complète en conditions opérationnelles.
- **Adversarial ML en Federated Learning** : explorer comment les attaques adversariales se comportent dans un contexte d’IDS distribué à apprentissage fédéré, perspective directement liée aux travaux en cours sur les réseaux IoT.

Message Final

La cybersécurité basée sur l’IA ne peut atteindre sa maturité opérationnelle que si ses praticiens intègrent l’Adversarial ML comme une composante incontournable du cycle de développement – non pas comme une curiosité académique, mais comme un **standard d’audit** au même titre que les tests de pénétration classiques. Ce projet illustre qu’un modèle excellent sur données propres peut être trivialement compromis par un attaquant informé, et que la robustesse s’acquiert, se certifie et se maintient de manière itérative.

Références

- [1] Department of Mechanical Engineering, MIT. *PGD-TO : A Scalable Alternative to MMA Using Projected Gradient Descent for Multi-Constraint Topology Optimization*. arXiv :2511.13905v1 [cs.CE] 17 Nov 2025.
- [2] J. Kennedy et R. Eberhart, « Particle swarm optimization » (Wikipédia), , IEEE International Conference on Neural Networks. *Optimisation par essaims particuliers*.